Contents

CNS Lab Answers and Explanations (Questions 1-40) **1**

1. Caesar cipher . . . 1
2. Monoalphabetic substitution cipher . . . 2
3. Playfair cipher . . . 2
4. Polyalphabetic substitution / Vigenere cipher . . . 2
5. Affine Caesar cipher . . . 2
6. Breaking the affine cipher from letter frequencies . . . 2
7. Simple-substitution cryptogram . . . 2
8. Keyword-based monoalphabetic cipher . . . 2
9. PT-109 Playfair decryption . . . 2
10. Encrypt with the supplied Playfair matrix . . . 3
11. Playfair keyspace size . . . 3
12. Hill cipher with key matrix [[9,4],[5,7]] . . . 3
13. Known-plaintext attack on a 2x2 Hill cipher . . . 3
14. One-time-pad version of Vigenere . . . 3
15. Additive-cipher frequency attack . . . 3
16. Monoalphabetic-substitution frequency attack . . . 3
17. DES decryption subkey order . . . 4
18. DES subkey halves . . . 4
19. CBC encryption with a cipher stronger than DES . . . 4
20. ECB/CBC error propagation . . . 4
21. Padding in ECB/CBC/CFB . . . 4
22. CBC with S-DES . . . 4
23. Counter mode with S-DES . . . 4
24. RSA private key for e=31, n=3599 . . . 4
25. RSA and a plaintext block sharing a factor with n . . . 5
26. RSA private-key leak and reuse of modulus . . . 5
27. RSA applied independently to each alphabetic character . . . 5
28. Diffie-Hellman protocol . . . 5
29. SHA-3 rate and capacity lanes . . . 5
30. CBC-MAC extension demonstration . . . 5
31. CMAC subkey generation . . . 5
32. DSA nonce and signature differences . . . 5
33. DES implementation . . . 6
34. Padding question repeated . . . 6
35. One-time-pad Vigenere stream . . . 6
36. Affine cipher (duplicate of Question 5) . . . 6
37. Monoalphabetic frequency attack (duplicate) . . . 6
38. Hill cipher known/chosen-plaintext attack (duplicate) . . . 6
39. Additive-cipher frequency attack (duplicate) . . . 6
40. Monoalphabetic frequency attack (duplicate) . . . 6

CNS Lab Answers and Explanations (Questions 1-40)

This answer key follows the numbering and wording of the supplied 5-page lab handout. Each question has a corresponding `qNN_*.c` source file. Because the handout sometimes phrases conceptual questions as "Write a C program," the corresponding files either run a demonstration or print the reasoning.

1. Caesar cipher

Use `q01_caesar.c`. Map each letter using `C = (P + k) mod 26`; decryption uses `P = (C - k) mod 26`. Example: `HELLO` with k=3 encrypts to `KHOOR`.

2. Monoalphabetic substitution cipher

Use `q02_monoalphabetic.c`. The user supplies a 26-letter permutation; every plaintext letter maps to one unique ciphertext letter. Decryption uses the inverse mapping. The program validates that each alphabet letter appears exactly once.

3. Playfair cipher

Use `q03_playfair.c`. It creates a 5x5 key square (I/J share one cell), splits plaintext into digraphs, inserts X between repeated letters and pads an odd-length message. Same-row pairs shift right for encryption, same-column pairs shift down, and rectangle pairs exchange columns.

4. Polyalphabetic substitution / Vigenere cipher

Use `q04_vigenere.c`. Each key character gives a shift: encryption is `C_i=(P_i+K_i) mod 26`, and decryption is `P_i=(C_i-K_i) mod 26`. The key repeats for ordinary Vigenere; a true one-time pad must use a random, secret, never-reused key stream.

5. Affine Caesar cipher

Use `q05_affine.c`. Encryption is `C=(aP+b) mod 26`. There is no special restriction on b; it is normalized modulo 26. The requirement on a is `gcd(a,26)=1`. Valid residues are `1,3,5,7,9,11,15,17,19,21,23,25`. The disallowed residues from 0 through 25 are `0,2,4,6,8,10,12,13,14,16,18,20,22,24`.

6. Breaking the affine cipher from letter frequencies

Use `q06_affine_frequency_attack.c`. Assuming the common English frequency guesses `E -> B` and `T -> U`, equations are `4a+b = 1 (mod 26)` and `19a+b = 20 (mod 26)`. Subtracting gives `15a=19 (mod 26)`, so a=3, then b=15. The inverse of 3 modulo 26 is 9. Decrypt by `P = 9(C-15) mod 26`. The supplied question does not include a separate ciphertext, so the program prompts for one; the frequency guesses may be wrong for a short or unusual message.

7. Simple-substitution cryptogram

Use `q07_substitution_cryptogram.c`. It counts cipher-symbol frequencies, provides interactive symbol-to-letter mapping, and prints the known solution to the classic Gold-Bug cryptogram:

> A good glass in the bishop's hostel in the devil's seat; forty-one degrees and thirteen minutes northeast and by north; main branch seventh limb east side; shoot from the left eye of the death's-head; a bee-line from the tree through the shot fifty feet out.

The question's non-ASCII glyphs are represented internally by aliases to keep the C source portable.

8. Keyword-based monoalphabetic cipher

Use `q08_keyword_substitution.c`. For keyword `CIPHER`, remove duplicate letters, write the keyword first, and append unused alphabet letters in order. The cipher alphabet is `CIPHERABDFGJKLMNOQSTU-VWXYZ`, matched against plaintext `ABCDEFGHIJKLMNOPQRSTUVWXYZ`.

9. PT-109 Playfair decryption

Use `q09_pt109_playfair_decrypt.c`. The key is `royal new zealand navy`; I/J share one square cell. The output includes digraph grouping and leaves spacing to be restored from context. The handout explicitly requires treating ciphertext TT as plaintext `tt`.

10. Encrypt with the supplied Playfair matrix

Use `q10_fixed_playfair.c`. The matrix is:

```
M F H I/J K
U N O P   Q
Z V W X   Y
E L A R   G
D S T B   C
```

The program encrypts the entered message using this fixed square, combines I/J, inserts X between duplicate pair letters, and pads an odd-length message.

11. Playfair keyspace size

Use `q11_playfair_keyspace.c`. With I/J combined there are 25 letters/cells, so raw permutations are `25!`. `log2(25!) ~= 83.6815`, giving about `2^83.7` keys. If global cyclic row/column translations are considered equivalent (25 layouts per class), the count is `25!/25 = 24!`, about `2^79.0`. State this equivalence assumption because some treatments count key squares differently.

12. Hill cipher with key matrix [[9,4],[5,7]]

Use `q12_hill_cipher.c`. Convert letters to 0-25 and encrypt each digraph as a column vector using `C=KP mod 26`. The determinant is `9*7 - 4*5 = 43 = 17 (mod 26)`, and the inverse of 17 modulo 26 is 23, so the matrix is invertible. Odd-length plaintext is padded with X. Use mode `E` to encrypt and `D` to decrypt.

13. Known-plaintext attack on a 2x2 Hill cipher

Use `q13_hill_known_plaintext_attack.c`. Arrange two known plaintext digraphs as columns of `P` and their ciphertext digraphs as columns of `C`; when `P` is invertible, recover the key as `K = C P^-1 (mod 26)`. The determinant must have a modular inverse modulo 26; otherwise choose different known blocks.

14. One-time-pad version of Vigenere

Use `q14_one_time_pad_vigenere.c`. It encrypts `SEND MORE MONEY` with the provided stream `9 0 1 7 23 15 21 14 11 11 2 8 9`. Removing spaces, the ciphertext is `BEOKJDMSXZPMH`. Part (b) computes the numeric key stream that maps the same ciphertext to `CASH NOT NEEDED`: `K_i=(C_i-P_i) mod 26`.

15. Additive-cipher frequency attack

Use `q15_additive_frequency_attack.c`. It tries all 26 possible shifts, scores each candidate against approximate English letter frequencies, and prints the ten highest-ranked candidates. It is a ranking heuristic; short messages may not produce the correct answer at rank 1.

16. Monoalphabetic-substitution frequency attack

Use `q16_monoalphabetic_frequency_attack.c`. It makes a frequency-based substitution guess and generates/ranks heuristic variants using common English bigrams and trigrams. This is educational rather than a guaranteed general solver: reliable automatic cracking usually needs a larger n-gram model, dictionary, or language score.

17. DES decryption subkey order

Use `q17_des_reverse_subkeys.c`. It generates the 16 DES subkeys from a 64-bit key (entered as 16 hex digits). DES decryption uses the same round structure but applies `K16, K15, ..., K1` in reverse order.

18. DES subkey halves

Use `q18_des_subkey_halves.c`. DES PC-1 forms 28-bit halves `C0` and `D0`. Each round rotates both halves; PC-2 selects 24 bits from `Cn` and 24 bits from `Dn` to form the 48-bit round subkey. The program prints the halves and the two 24-bit portions for each round.

19. CBC encryption with a cipher stronger than DES

Use `q19_cbc_aes.c`. The demonstration uses AES-256-CBC (OpenSSL) to encrypt and decrypt entered text. AES is the recommended modern choice over DES/3DES for new systems. If comparing just DES and 3DES, 3DES is stronger than single DES but substantially slower and legacy; for modern security and performance, prefer a vetted AES mode. The program's fixed demo key/IV are not production-safe, and CBC alone does not authenticate ciphertext.

20. ECB/CBC error propagation

Use `q20_ecb_cbc_error_propagation.c`. In CBC, a bit error in transmitted `C1` corrupts the decrypted `P1` block and flips the corresponding bit in `P2`; later blocks are not affected by that isolated ciphertext error. A bit error in source `P1` changes `C1`, and because CBC chains ciphertext blocks, it propagates through all following ciphertext blocks; at the receiver, `P1` and subsequent plaintext blocks are affected. ECB damage is limited to the matching block.

21. Padding in ECB/CBC/CFB

Use `q21_padding_demo.c`. The program appends a `1` bit and enough `0` bits to finish a block. Padding every message, even when the plaintext already fills a block, gives the receiver an unambiguous padding marker and avoids confusion between real trailing data and padding. The padding rule here demonstrates the assignment's idea; production protocols should use standardized framing and authentication.

22. CBC with S-DES

Use `q22_sdes_cbc.c`. It implements the 10-bit S-DES key schedule, 8-bit block encryption/decryption, and CBC with IV `10101010` and key `01111 11101`. For the plaintext blocks `00000001 00100011`, the generated ciphertext is `11110100 00001011`; decryption recovers the input.

23. Counter mode with S-DES

Use `q23_sdes_ctr.c`. The counter begins at `00000000` and increments once per 8-bit block. For plaintext `00000001 00000010 00000100` and key `01111 11101`, the expected ciphertext is `00111000 01001111 00110010`. CTR uses the same XOR operation for encryption and decryption.

24. RSA private key for e=31, n=3599

Use `q24_rsa_private_key.c`. Factor `3599 = 59 * 61`. Then `phi(n)=(59-1)(61-1)=3480`. The modular inverse of 31 modulo 3480 is `d=3031`. Private key: `(d,n)=(3031,3599)`.

25. RSA and a plaintext block sharing a factor with n

Use `q25_rsa_common_factor.c`. If a plaintext block `m` shares a nontrivial factor with `n`, `gcd(m,n)` reveals that factor. Since `c=m^e mod n`, a corresponding ciphertext block also shares the factor, so `gcd(c,n)` can reveal it even when the numeric plaintext is unknown. Knowing only an abstract claim without the corresponding ciphertext/block does not by itself identify the factor.

26. RSA private-key leak and reuse of modulus

Use `q26_rsa_key_reuse.c`. It is not safe to keep the same modulus after the private key leaks. Knowledge of `d`, `e`, and `n` can enable factor recovery, and old ciphertexts remain compromised. Generate new primes and a new modulus, replace the key pair, and revoke the old key.

27. RSA applied independently to each alphabetic character

Use `q27_rsa_small_alphabet_attack.c`. Raw RSA is deterministic and the plaintext domain is only 26 values. An attacker can compute `m^e mod n` for every `m` from 0 through 25 and make a codebook matching each ciphertext to a letter. Use secure randomized padding such as RSA-OAEP for RSA encryption rather than encrypting characters directly.

28. Diffie-Hellman protocol

Use `q28_diffie_hellman.c`. With prime `p` and generator `g`, Alice sends `g^x mod p`, Bob sends `g^y mod p`; both calculate `g^(xy) mod p`. If they instead exchange `x^g` and `y^g`, then `(y^g)^x` and `(x^g)^y` are not generally equal, so it does not form the same shared-key protocol. Real deployments need large standardized parameters and authentication.

29. SHA-3 rate and capacity lanes

Use `q29_sha3_capacity_lanes.c`. A 1600-bit Keccak state with a 1024-bit rate has a 576-bit capacity. With 64-bit lanes, that is 16 rate lanes and 9 capacity lanes. Under the instruction to ignore the permutation, XORing blocks only touches the rate portion, so the original zero capacity lanes stay zero forever. The actual permutation is the diffusion mechanism that would mix state values.

30. CBC-MAC extension demonstration

Use `q30_cbc_mac_forgery.c`. For one-block message `X`, let `T=E_K(X)`. For two-block input `X || (X XOR T)`, the second CBC input is `(X XOR T) XOR T = X`, so its MAC is again `E_K(X)=T`. The code uses a tiny toy block transform solely to illustrate this equality, not to provide real security. This is why basic CBC-MAC needs fixed-length messages or proper length/domain binding.

31. CMAC subkey generation

Use `q31_cmac_subkeys.c`. For a 64-bit block, the reduction constant is `Rb=0x1B`; for a 128-bit block, it is `Rb=0x87`. Shift `L=E_K(0^n)` left by one bit to make K1, XORing Rb into the least-significant byte if the original most-significant bit was 1. Repeat using K1 to get K2.

32. DSA nonce and signature differences

Use `q32_dsa_nonce_demo.c`. The demonstration signs the same hash twice with distinct nonce values, so the signatures differ. DSA requires a fresh, secret nonce; nonce reuse or leakage can reveal the private key. Textbook RSA signatures are deterministic for the same hash/key, though real RSA signature schemes use standardized encodings.

33. DES implementation

Use `q33_des_full.c`. This program uses OpenSSL's DES implementation for one 64-bit block, prints ciphertext hex, and decrypts to recover the padded input. DES has a 56-bit effective key and is obsolete/insecure; use only for the lab exercise, not production protection.

34. Padding question repeated

Use `q34_padding_demo.c`. Same as Question 21: append a 1 marker and then zero bits to reach a block boundary. Adding padding even for an already complete block provides a consistent, unambiguous end marker.

35. One-time-pad Vigenere stream

Use `q35_otp_vigenere_stream.c`. The program asks for E/D, text, and one integer from 0 to 25 per alphabetic character. Encryption adds the stream value modulo 26; decryption subtracts it. A secure one-time pad needs uniformly random, secret, message-length key material that is never reused.

36. Affine cipher (duplicate of Question 5)

Use `q36_affine_duplicate.c`. It applies `C=(aP+b) mod 26` and checks `gcd(a,26)=1`. `b` is unrestricted modulo 26.

37. Monoalphabetic frequency attack (duplicate)

Use `q37_monoalphabetic_attack.c`. It uses the same frequency-based, common-n-gram heuristic as Question 16. The ranking is approximate and may need human inspection.

38. Hill cipher known/chosen-plaintext attack (duplicate)

Use `q38_hill_attack_duplicate.c`. It solves a 2x2 Hill key with `K=C P^-1 mod 26` when the known plaintext matrix is invertible modulo 26.

39. Additive-cipher frequency attack (duplicate)

Use `q39_additive_attack_duplicate.c`. It tries all 26 shifts and returns the top 10 likelihood-scored plaintexts.

40. Monoalphabetic frequency attack (duplicate)

Use `q40_monoalphabetic_attack_duplicate.c`. It uses the same heuristic approach as Questions 16 and 37, with the important limitation that frequency heuristics cannot guarantee decryption of every arbitrary substitution cipher.